

LAPORAN CODE REVIEW

Mata Kuliah Rekayasa Penjamin Kualitas Sistem Informasi

Kelompok 3

Muhammad Irvan Fadly, Egi Dea Ayu Safitri, Diffa Muhammad Al Farisyi, Saraswaty Dwi Andiny, Leonardo Diego Tristan

Laporan Defect

Berikut adalah daftar defect yang ditemukan pada source code fitur Todo App berbasis Flask, beserta kategori, rekomendasi perbaikan, dan status penyelesaiannya.

No	Defect	Kategori	Rekomendasi	Status
1	Tidak ada validasi input pada endpoint POST /todos. Aplikasi crash (KeyError) jika field 'title' tidak dikirim dalam request body.	Input Validation Error	Tambahkan pengecekan kondisi sebelum memproses data: if not data or 'title' not in data or not data['title'].strip(). Kembalikan HTTP 400 Bad Request jika kondisi tidak terpenuhi.	Fixed
2	Objek datetime Python tidak bisa di-serialize ke JSON secara langsung, menyebabkan TypeError saat response dikirim ke client.	Type Error / Serialization	Gunakan .isoformat() pada datetime.now() untuk mengkonversi ke format string ISO 8601 yang kompatibel dengan JSON, contoh: datetime.now().isoformat().	Fixed
3	Logika update pada endpoint PUT /todos/<id> selalu men-set done=True secara	Logic Error	Ambil nilai done dari request body (data['done']) dan update secara kondisional menggunakan if	Fixed

No	Defect	Kategori	Rekomendasi	Status
	hardcode. Pengguna tidak bisa mengubah status kembali ke False.		'done' in data. Tambahkan juga dukungan update field title.	
4	Endpoint DELETE /todos/<id> tidak mengembalikan response apapun. Flask error jika todo_id tidak ditemukan karena fungsi mereturn None.	Runtime Error / Missing Response	Tambahkan return jsonify({'message': '...'}), 200 untuk sukses dan return jsonify({'error': 'Not found'}), 404 untuk kasus ID tidak ditemukan.	Fixed

Ringkasan:

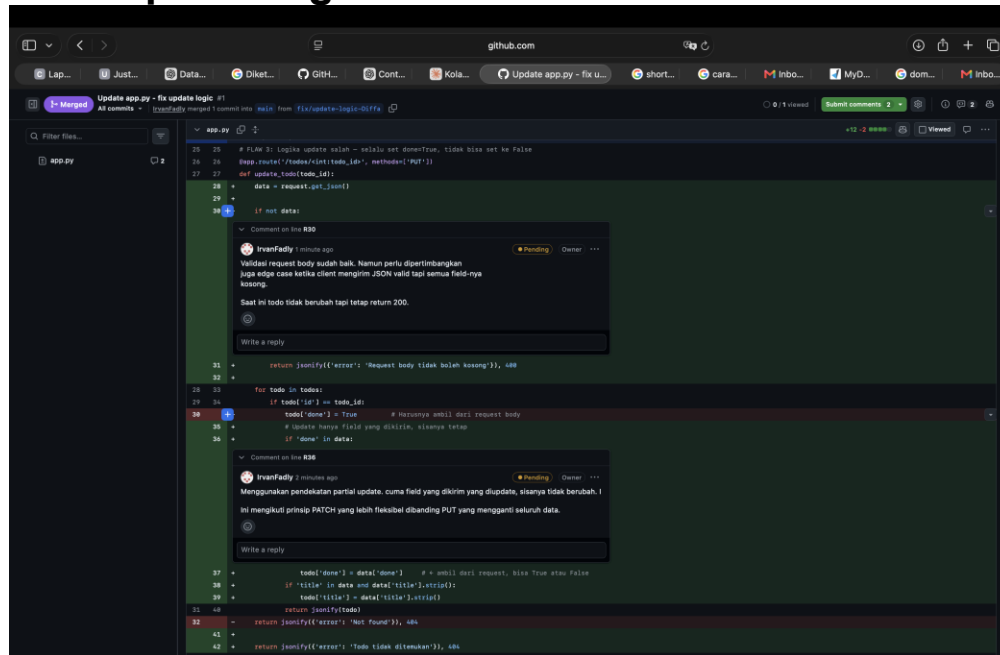
Total defect ditemukan : 4

Total defect diperbaiki : 4 (100%)

Status keseluruhan : Semua defect telah diselesaikan

Screenshot komentar reviewer & bukti perubahan setelah review

1. Fix update logic



The screenshot shows a GitHub pull request interface. The title of the pull request is "Update app.py - fix update logic". The pull request is from the user "IvanFady" and is targeting the "main" branch. The pull request is currently in a "Pending" state. The code changes are visible in the "app.py" file. The changes include a new function "update_todo" and a new endpoint "/update_todo". The code is written in Python and uses Flask for the web framework. The code is as follows:

```
25 # Route 2: Logic update todo - validasi set done/False, tidak bisa set ke False
26 @app.route('/update_todo_id', methods=['PUT'])
27 def update_todo_id():
28     data = request.get_json()
29     +
30     if not data:
31         return jsonify({'error': 'Request body tidak boleh kosong'}), 400
32     +
33     for todo in todos:
34         if todo['id'] == todo_id:
35             todo['done'] = True # Harusnya ambil dari request body
36             # Update hanya field yang dikirim, lainnya tetap
37             if 'done' in data:
38                 +
39                 todo['done'] = data['done'] # = ambil dari request, bisa True atau False
40                 if 'title' in data and data['title'].strip():
41                     todo['title'] = data['title'].strip()
42                 return jsonify(todo)
43     -
44     return jsonify({'error': 'Not found'}), 404
45     +
46     return jsonify({'error': 'Todo tidak ditemukan'}), 404
```

There are two comments from IvanFady on the pull request. The first comment is on line 30 and says: "Validasi request body sudah baik. Namun perlu dipertimbangkan juga edge case ketika client mengirim JSON valid tapi semua field-nya kosong. Saat ini todo tidak berubah tapi tetap return 200." The second comment is on line 36 and says: "Menggunakan pendekatan partial update, cuma field yang dikirim yang diupdate, lainnya tidak berubah. Ini mengikuti prinsip PATCH yang lebih fleksibel dibanding PUT yang mengganti seluruh data."

2. Fix datetime serial

The screenshot shows a GitHub pull request titled "Update app.py - fix datetime serial #2" with commit hash 8e5e98a. The pull request is merged and authored by Egeida 18 hours ago. The file being modified is `app.py`. The code shows a function `create_todo()` that creates a new todo item. The original code (lines 15-19) had a `'created_at': datetime.now()` line, which was flagged as a "FLAW 2: datetime tidak bisa di-serialize ke JSON". The updated code (lines 15-19) changes this to `'created_at': datetime.now().isoformat()`, with a comment indicating it's changed to ISO 8601 string format. A comment by IrvanFadly explains that `.isoformat()` is correct for JSON serialization, while `.strftime()` is an alternative.

```
12 12 def create_todo():
13 13     data = request.get_json()
14 14     global next_id
15 15     todo = {
16 16         'id': next_id,
17 17         'title': data['title'], # KeyError jika 'title' tidak ada
18 18         'done': False,
19 19         'created_at': datetime.now() # FLAW 2: datetime tidak bisa di-serialize ke JSON
20 20     }
21 21     todo = {
22 22         'id': next_id,
23 23         'title': data['title'],
24 24         'done': False,
25 25         'created_at': datetime.now().isoformat() # * diubah jadi string format ISO 8601
26 26     }
```

Comment on line 19

IrvanFadly now

Penggunaan `.isoformat()` sudah benar untuk serialisasi datetime ke JSON. Alternatif lain yang bisa dipertimbangkan adalah menggunakan `strftime()`.

Write a reply

Resolve comment

3. Fix delete response

The screenshot shows a GitHub pull request titled "Update app.py - delete fix #3" which has been merged. The commit history shows two commits by IrvanFadly: one merging commit 51aee92 into main 18 hours ago, and another changing the title 9 minutes ago. A review by IrvanFadly is shown, with a comment on the code change in `app.py`. The code change shows a modification to the `delete` function in `app.py`, adding a response body for the 404 status code. The review comment states: "Response body sudah ditambahkan dengan baik." (The response body has been added correctly). A note mentions that the REST API DELETE method can use status code 204 (No Content) or 200 with a response body. The pull request is now closed, and the branch `fix/delete-response-saras` can be safely deleted.

Update app.py - delete fix #3
IrvanFadly merged 2 commits into main from fix/delete-response-saras

IrvanFadly merged commit 51aee92 into main 18 hours ago

IrvanFadly changed the title Update app.py Update app.py - delete fix 9 minutes ago

IrvanFadly reviewed 1 minute ago

app.py

```
50 50         return jsonify({'message': f'Todo {todo_id} berhasil dihapus'}), 200 # +  
51 51  
52 -     return jsonify({'error': 'Todo tidak ditemukan'}), 404 # + handle kalau id tidak  
52 +     return jsonify({'error': 'Todo tidak ditemukan'}), 404 # + handle kalau id tidak
```

IrvanFadly 4 minutes ago

Response body sudah ditambahkan dengan baik.

Catatan: pada beberapa konvensi REST API, method DELETE biasanya bisa menggunakan status code 204 (No Content) jika tidak mengembalikan response body. Tapi penggunaan status code 200 dengan pesan konfirmasi juga tetap valid

Pull request successfully merged and closed
You're all set — the `fix/delete-response-saras` branch can be safely deleted.

4. Fix validation input

The screenshot shows a GitHub pull request interface. The title is "Update app.py - validation fix #4". The pull request is from the repository "fix/input-validation-leonardo" to the "main" branch. It shows 2 commits merged into the main branch. The file "app.py" is selected, showing a diff with 11 additions and 6 deletions. The code includes a validation check for the 'title' field, ensuring it is not empty and returning a 400 status code if invalid. Two comments by user "IrvanFadly" are visible, discussing the validation logic and the status code.

Update app.py - validation fix #4
All commits · leonardodasilva merged 2 commits into main from fix/input-validation-leonardo · 0 / 1 viewed · Submit comments 2 · Viewed

app.py 2

```
15 -     todo = {  
16 -     'id': next_id,  
17 -     'title': data['title'],  
18 -     'done': False,  
19 -     'created_at': datetime.now().isoformat() # + diubah jadi string format ISO 8601  
20 - }  
15 +  
16 + # Validasi: data harus ada, title wajib ada dan tidak boleh kosong  
17 + if not data or 'title' not in data or not data['title'].strip():  
18 +  
19 +     return jsonify({'error': 'Title wajib diisi dan tidak boleh kosong'}), 400  
20 +  
21 +     return jsonify(todo), 201
```

Comment on line R17

IrvanFadly 30 minutes ago · Pending · Owner · ...

Validasi sudah tepat. Pengecekan bertahap lebih aman daripada langsung akses data['title']. Good job.

Write a reply

Comment on line R18

IrvanFadly 28 minutes ago · Pending · Owner · ...

Status code 400 (Bad Request) sudah sesuai untuk kasus invalid.

Write a reply

Refleksi

Bagian ini membahas dampak proses code review yang telah dilakukan terhadap kualitas dan maintainability kode proyek Todo App.

1. Dampak terhadap Kualitas Kode

Proses code review yang dilakukan secara kolaboratif melalui GitHub Pull Request berhasil mengidentifikasi empat defect yang tidak terdeteksi selama fase pengembangan awal. Defect-defect ini meliputi kesalahan validasi input, error serialisasi tipe data, logic error pada fitur update, dan missing response pada endpoint delete.

Tanpa proses review, keempat defect tersebut berpotensi menyebabkan aplikasi crash di lingkungan production, memberikan respons yang tidak konsisten kepada pengguna, serta menyulitkan proses debugging karena tidak adanya pesan error yang informatif.

Setelah perbaikan dilakukan, setiap endpoint kini menangani edge case dengan benar, mengembalikan HTTP status code yang sesuai, dan memberikan pesan error yang jelas kepada client.

2. Dampak terhadap Maintainability Kode

Perbaikan yang dihasilkan dari proses review membuat kode lebih mudah dipahami, dikembangkan, dan dipelihara oleh anggota tim lain. Penambahan validasi input dan penanganan error yang konsisten membuat perilaku API lebih dapat diprediksi.

Penggunaan branch per fitur dan Pull Request di GitHub juga memaksa setiap anggota untuk mendokumentasikan alasan di balik setiap perubahan melalui commit message dan deskripsi PR. Hal ini menciptakan riwayat perubahan kode yang terjaga dengan baik dan memudahkan audit di masa depan.

Selain itu, pemisahan tanggung jawab (satu anggota mengerjakan satu flaw) melatih kebiasaan kerja modular yang penting dalam pengembangan perangkat lunak skala besar.